

1. ArrayBasics.java

* `int[] a = {10, 20, 30, 40, 0, 0, 0, 0}`

10	20	30	40	0	0	0	0
----	----	----	----	---	---	---	---

① Insert : `a[1] = 15`
 at index 1

10	15	20	30	40	0	0	0
----	----	----	----	----	---	---	---

shifts = 3

② Insert at ^{ending} index :

10	15	20	30	40	50	0	0
----	----	----	----	----	----	---	---

shifts = 0

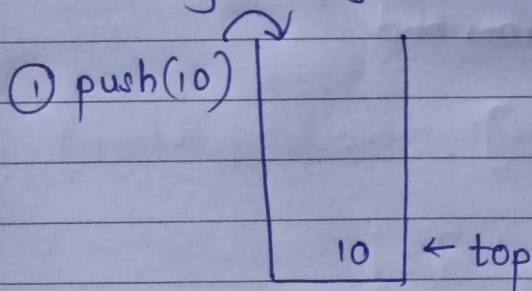
③ Delete at index 0

15	20	30	40	50	0	0	0
----	----	----	----	----	---	---	---

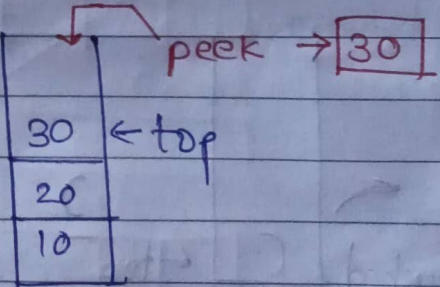
shifts = 5

* ~~2D Array~~

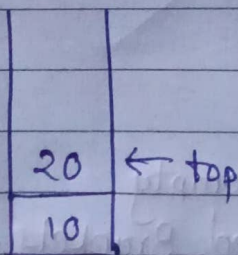
2. ArrayStack.java



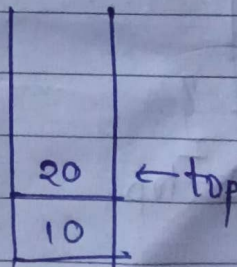
③ push(30)



② push(20)



④ pop(30)



⑤ push(40, 50)

50	← top
40	
20	
10	

Stack Full ✓

~~Aftering~~ After removing all the elements [LIFO]

⇒ Stack Underflow

3. Balanced Brackets.java

1] $a[i] = (b+c) * [d]$

[] () { }

opening brackets → pushed into the ~~stack~~

stack

pop ←

	<u>]</u>
pushed	<u>[</u> ← top

* if '[' matches ']' → pop

After a closing bracket is matched with the top element & it is its corresponding opening bracket; the opening bracket is popped.

pop ←

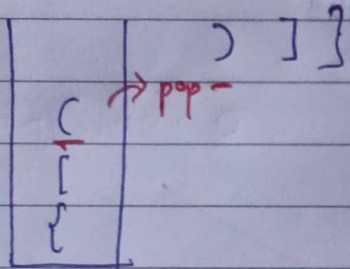
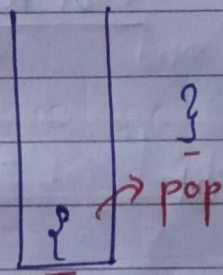
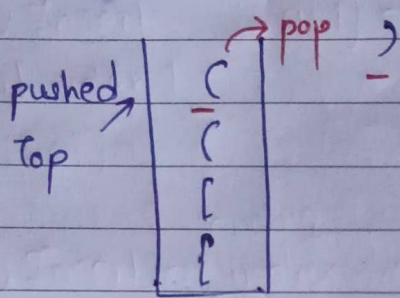
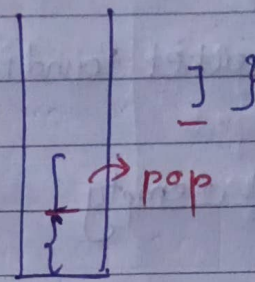
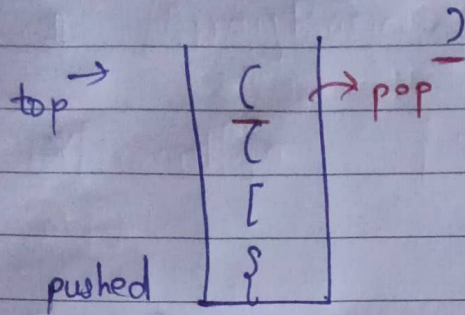
	<u>)</u>
pushed	<u>(</u> ← top

pop ←

	<u>}</u>
pushed	<u>{</u> ← top

Now Stack is empty
Hence, Balanced Brackets.

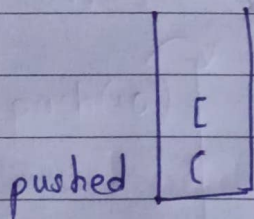
2) {[(())]}



Stack Empty ✓

Hence, Brackets are balanced.

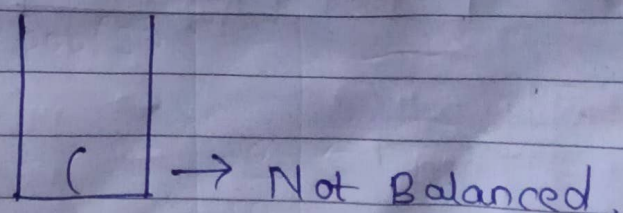
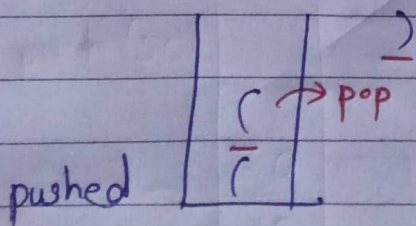
3) ([)]



Not found the corresponding closing bracket.

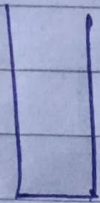
Not Balanced

4) ((a+b))



5) a+b)

No opening bracket found for)



Stack is empty

Not Balanced

6)

"" Null input

There is no incomplete bracket pairs

Hence, balanced

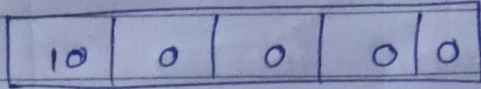
4. Linear Queue, java

①

front = 0, rear = -1

Enqueue

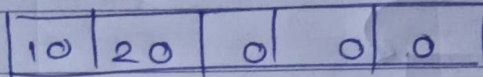
10, 20, 30, 40, 50



front = 0

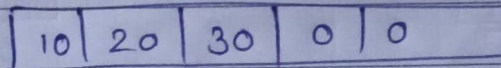
rear = 0

← Insert from rear end



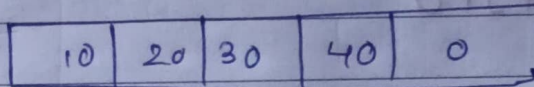
front = 0

rear = 1



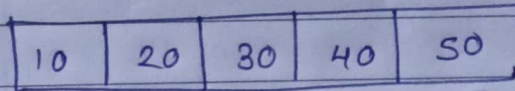
front = 0

rear = 2



front = 0

rear = 3



front = 0

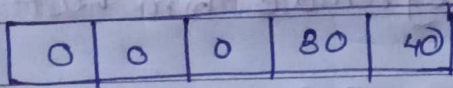
rear = 4

②

Dequeue 10, 20, 30

③

Enqueue 60



front = 3

rear = 4

rear = max - 1

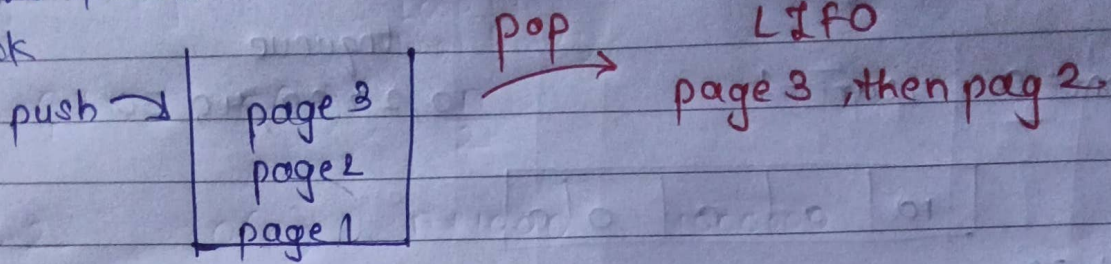
Queue Overflow

But 3 slots are empty

Solution: Circular queue

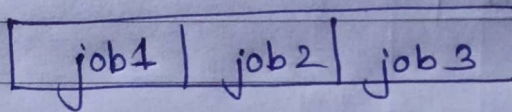
5. Java Queues Demo

① Stack



② Queue

offer at the rear end



← poll from the front end

job 1 then job 2

FIFO

③ Dequeue

Both ends

dg.offerLast(2) [2]

dg.offerFirst(1) [1, 2]

dg.offerLast(3) [1, 2, 3]

→ [Dequeue] remove from Last

dg.pollLast: [1, 2]

dg.pollFirst: → Removed [2]

④ Operations on empty queue

empty.poll() → null

empty.peek() → null

empty.remove() → Throws an exception, as there is no element in the queue to remove.

⑤ `ArrayDeque.offer(null)` forbids null
↳ `NullPointerException`

⑥ Bounded Circular Queue
Capacity : 2 Initial state : []

`bounded.offer(1)` [1]

`bounded.offer(2)` [1, 2]

Capacity is full

`bounded.offer(3)` → Returns false